ProcRoute: Process-Scoped Authorization of Split-Tunnel Routes

Arul Thileeban Sagayam
Bloomberg
New York, USA
asagayam@bloomberg.net

Abstract

In most split-tunnel VPN/ZTNA deployments, installing an internal route authorizes the entire device, not a specific application, to use it. An unprivileged malicious process can therefore reach internal services by reusing routes intended for corporate applications.

We present **ProcRoute**, a system that restricts internal-route access to explicitly authorized applications. ProcRoute models route access as an access-control problem: application identities are principals, destination prefixes with port and protocol constraints are resources, and a total, default-deny decision function mediates every `connect()` and UDP `sendmsg()` to an internal destination. Processes without a grant retain external access but are denied internal routes under our threat model.

We describe ProcRoute's formal model, a Linux prototype built on cgroup v2 and eBPF socket-address hooks, and two complementary evaluations. In a two-machine WireGuard deployment, ProcRoute matches the WireGuard baseline and 13% faster than an nftables cgroup-matching configuration, with a p50 connect latency of 93 µs (+3.6 µs over baseline), flat policy scaling to 5 000 prefixes, and sub-millisecond revocation. Single-machine loopback microbenchmarks confirm low hook overhead: 2.7 µs on the internal-allow path, 82/82 unauthorized pivot attempts blocked, and zero transient allows across 1.2 million connection attempts during policy reload.

CCS Concepts

• **Security and privacy → Network security**;

Keywords

remote access, split tunneling, eBPF, VPN, zero trust

ACM Reference Format:
Arul Thileeban Sagayam. 2026. ProcRoute: Process-Scoped Authorization of Split-Tunnel Routes. In *Proceedings of 31st ACM Symposium on Access Control Models and Technologies (SACMAT '26).* ACM, New York, NY, USA, 12 pages.

1 Introduction

Remote work has made remote access VPNs and ZTNA agents default infrastructure. Many organizations enable split tunneling so endpoints can access local networks and the public Internet efficiently while still reaching internal resources. However, split tunneling weakens the boundary between "trusted" corporate routes and "untrusted" external networks: the endpoint becomes a bridge, and internal routes are trivially reachable by any local process [7]. Split tunneling persists in most enterprise deployments despite guidance discouraging it [5, 8, 9], because full-tunnel routing imposes unacceptable latency, capacity costs, and user friction at scale; yet no standard mechanism on commodity desktop OSes restricts which processes may use VPN-installed routes.

This is well-understood. Guidance around VPN deployments explicitly calls out the risk of split-tunnel configurations that allow a host to communicate to the Internet while simultaneously connected to organizational resources, since the external connection can be used to attack or infiltrate the VPN-connected environment [8]. Nevertheless, in practice enterprises use split tunneling for usability and throughput, particularly when only a subset of applications truly need internal access.

We argue that the core issue is *granularity*. Enterprise remote access makes a decision at the device level, thus allowing routes to internal prefixes, and the operating system then allows any application to use those routes. Many endpoint threats manifest as *separate unprivileged processes* (phishing-launched payloads, malicious installers, adware bundles, and commodity malware running as the user) that are distinct from any legitimate corporate application yet inherit the same VPN routes. A second class of threats, in-process code execution via browser extensions, library injection, or memory corruption, operates within an already-running application and is not distinguishable at process granularity; we scope this class out of ProcRoute's threat model and analyze it as a limitation later. For the separate-process class, route authorization at process granularity is a feasible enforcement boundary that current split-tunnel deployments do not provide.

ProcRoute introduces process-scoped route authorization: only processes bound to an authorized principal may reach internal prefixes. Other processes are denied access to internal routes but are not restricted on external destinations, preserving split-tunnel Internet access. ProcRoute is designed to coexist with existing split-tunnel configurations and to impose low overhead on authorized connections.

1.1 Contributions

Commodity OSes provide basic packet-filtering primitives, but they don't treat VPN installed routes as resources, and no existing tool provides formal policy semantics for this enforcement. ProcRoute addresses this gap with three contributions: (1) A *formal access-control model* that treats application identities as principals, destination prefixes as resources, and enforces default-deny through a total decision function with six stated invariants, enabling security guarantees (e.g., monotone composition, non-widening updates) that cannot be stated for ad-hoc filter configurations (Section 3). (2) *Composable enforcement*: attaching the BPF program

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
SACMAT '26, Waterloo, Canada
© 2026 Copyright held by the owner/author(s).

with BPF_F_ALLOW_MULTI yields monotone composition: descendant cgroups can restrict but never expand the ancestor's allow set (Section 3.7). (3) A *split client/gateway architecture* for WireGuard, evaluated on a two-machine deployment and single-machine loopback microbenchmarks (Sections 4–6, Appendix A).

1.2 Delta vs. nftables and systemd

Existing tools(nftables, systemd) can approximate per-process filtering, but they are low-level packet-filter primitives. ProcRoute adds formal policy semantics, monotone composition, structured deny auditing, and optional per-binary hash verification (detailed in Sections 3–3.10).

2 Threat Model and Goals

2.1 Threat model

Procroute is designed to reduce risk when an adversary obtains code execution as a *separate, unprivileged process* [7]:

- **Adversary:** a process running as the user that is *distinct* from any authorized corporate application, for example a phishing-launched payload or adware executing as its own process.
- **Objective:** laterally move into internal networks by using VPN-installed routes.
- **Assumptions:** the OS kernel and ProcRoute enforcement are not compromised; the adversary does not have root/admin privileges sufficient to disable system security policy or load competing eBPF programs.

We explicitly **do not** claim to defend against:

- a kernel-level adversary or a local admin who can remove the enforcement agent, or
- *in-process code execution* within an already-authorized application (e.g., a malicious browser extension or exploitation of a memory-corruption vulnerability in an authorized binary). In-process compromise inherits the victim process's cgroup membership and therefore its network grants; ProcRoute cannot distinguish legitimate from injected code within a single address space. This limitation is inherent to any enforcement boundary drawn at process granularity.

2.2 Goals

ProcRoute is designed around four goals:

(1) **Process-level least privilege:** internal routes should be usable only by explicitly authorized applications.
(2) **Split-tunnel compatible:** preserve direct Internet access for the rest of the system and avoid forcing all traffic through a VPN gateway.
(3) **Low overhead:** enforcement should add minimal latency/CPU overhead for allowed flows.
(4) **Auditable decisions:** policy should be readable and decisions should be loggable for incident response and compliance.

3 ProcRoute Access-Control Model

We now formalize ProcRoute as an access-control system, defining principals, resources, the policy language, the enforcement decision function, and the security invariants the system must maintain.

ProcRoute supports two deployment modes that share the same principal-binding and grant-evaluation machinery but differ in where the deny verdict is applied:

- **Local-only enforcement.** Cgroup socket-address hooks evaluate the decision function directly on the endpoint, returning EPERM for unauthorized connections (Sections 3.5–3.9).
- **Split client/gateway enforcement.** The same cgroup hooks resolve the originating principal on the client but *tag* the socket instead of denying; a cooperating gateway enforces per-peer policy over a WireGuard tunnel (Section 3.6).

Both modes may operate simultaneously: local enforcement provides immediate EPERM feedback, while gateway enforcement adds a second, network-level enforcement point.

3.1 Principals

A *principal* in ProcRoute is an application identity $p \in \mathcal{P}$, where $\mathcal{P}$ is a finite set of identifiers defined by administrative policy. Each principal corresponds to a logical application that may comprise one or more OS processes (e.g., a browser and its helper processes).

Principal binding is realized through cgroup membership. Let $\mathcal{C}$ denote the set of all cgroup nodes in the hierarchy. The ProcRoute controller creates a dedicated cgroup subtree for each $p \in \mathcal{P}$ and moves the application's processes into it. We define two total functions:

- $cg : PID \to \mathcal{C}$ maps every live process to its unique cgroup,
- $bind : \mathcal{C} \to \mathcal{P} \cup \{\bot\}$ maps each cgroup to its bound principal, or to $\bot$ if the cgroup has no binding. Only finitely many cgroups map to a value in $\mathcal{P}$; all others map to $\bot$.

The effective principal of a process is the composition $prin(pid) = bind(cg(pid))$. Because both cg and $bind$ are total, $prin$ is total: every process has a well-defined principal (or $\bot$).

A process whose cgroup has no binding ($prin(pid) = \bot$) is *unauthorized* and receives no grants. This is the common case: the vast majority of cgroups on a system are unbound, and the vast majority of processes are therefore unauthorized with respect to ProcRoute.

Policy updates. $\mathcal{P}$, $\mathcal{I}$, and $\mathcal{R}$ are parameters of a policy *instance*. The controller may atomically replace the contents of the BPF maps to install a new policy instance (e.g., when the VPN session reconnects with different route grants). The model treats each instance as a static configuration; transitions between instances are handled by the controller and fall outside the formal model here.

Revocation semantics. Revoking access for principal p at time t involves two guarantees of decreasing strength:

- **G1 (connect-time):** Every connect() or first sendmsg() call issued by p after time t to a destination outside R'_p (the updated grant set) is denied.
- **G2 (data-path):** Every *existing* socket opened by p to a destination in $R_p \setminus R'_p$ is terminated within a bounded interval δ after time t.

G1 is immediate whenever the BPF maps are updated, because each hook re-evaluates the policy on every call (Section 3.5). G2 requires an *authorization epoch*: the controller increments a global epoch counter ε in a BPF array on each policy update. Sockets authorized at epoch $\varepsilon' < \varepsilon$ are *stale*. The prototype implements two revocation paths for stale sockets: (a) the `sendmsg` hook checks the epoch recorded per socket cookie and denies further sends (immediate for UDP); (b) a background sweeper terminates stale TCP sockets via `ss -kill` (`SOCK_DESTROY`) at a configurable interval.

3.2 Resources

A *resource* is a network destination reachable via a split-tunnel route. Each resource is a tuple

$$r = \langle prefix,\ proto,\ [port_lo,\ port_hi] \rangle$$

where *prefix* is an IPv4 or IPv6 CIDR prefix, $proto \in \{\text{TCP, UDP}, *\}$ specifies the transport protocol ($*$ matches both TCP and UDP), and $[port_lo,\ port_hi]$ is an inclusive port range. The sentinel value $[0, 0]$ denotes "all ports": when $port_lo = 0$, the port constraint is vacuously satisfied regardless of the destination port. The protocol and port wildcards compose independently: $\langle prefix,\ *,\ [0, 0] \rangle$ covers every TCP and UDP connection to any port within *prefix*.

Resources are *flat*: there is no hierarchy or subsumption among them; the policy is simply a set of grants. Overlapping prefix ranges are not given any special precedence in the policy language (coverage is defined by the covering relation below). For efficient kernel enforcement via a single LPM lookup, ProcRoute additionally assumes a well-formed, destination-disjoint normalized grant set.

Overlaps and LPM resolution. The formal decision function permits multiple grants in R_p to cover the same destination tuple, yielding a union-of-grants semantics. The kernel prototype uses a per-principal longest-prefix-match (LPM) trie for efficiency (Section 3.8) and therefore obtains at most one prefix match for a given destination IP. To reconcile these, ProcRoute assumes (and the controller can validate) a *well-formed* grant set: after normalizing duplicates, each principal's resources are *destination-disjoint*: for any destination IP d, at most one $r \in R_p$ has $d \in r.prefix$. Overlapping entries with identical protocol/port constraints are merged; overlaps with incompatible constraints are rejected as ambiguous. This rules out policies that assign different port constraints to overlapping prefixes for the same principal (e.g., `10.0.0.0/8` on port 443 and `10.1.0.0/16` on port 22); enterprise firewall audits suggest 15–25 % of zone-based rulesets contain such entries [19, 20]. A *cascaded trie* extension that lifts this restriction with one additional map lookup is described in Appendix A.6. Under the destination-disjoint condition, the existential check $\exists r \in R_p : dst \sqsubseteq r$ is equivalent to a single LPM lookup followed by the matched rule's protocol/port predicate.

We write $dst \sqsubseteq r$ to indicate that a concrete connection attempt with destination IP d, protocol t, and port q is *covered* by resource r:

$$\begin{aligned} dst \sqsubseteq r \iff\ & d \in r.prefix \ \wedge\ (r.proto = * \vee t = r.proto) \\ & \wedge\ (r.port_lo = 0 \vee r.port_lo \le q \le r.port_hi) \end{aligned}$$

3.3 Internal route set

The administrator defines a finite set of internal prefixes $\mathcal{I} = \{i_1, i_2, \ldots, i_n\}$, each an IP CIDR block. A destination IP d is *internal* iff $d \in \bigcup_{i \in \mathcal{I}} i$. Destinations outside $\mathcal{I}$ are considered external and are not subject to ProcRoute enforcement; they pass through unconditionally to preserve split-tunnel Internet access.

3.4 Policy

A ProcRoute policy Π is a triple $\Pi = \langle \mathcal{I},\ \mathcal{P},\ \mathcal{R} \rangle$, where:

- $\mathcal{I}$ is the internal route set (set of CIDR prefixes),
- $\mathcal{P}$ is the set of application principals, and
- $\mathcal{R} : \mathcal{P} \to 2^{Resource}$ assigns each principal a (possibly empty) set of allowed resources.

We write $R_p = \mathcal{R}(p)$ for the allow set of principal p.

ProcRoute's policy language is *purely positive*: grants authorize access; there are no deny rules, exceptions, or negative grants. A principal's effective access is exactly $\bigcup_{r \in R_p} \{dst \mid dst \sqsubseteq r\}$, the union of destinations covered by its grants. There is no mechanism to deny a sub-range within a broader grant; restricting a principal's access requires narrowing its grant set.

The policy is *default-deny*: this follows directly from the structure of the decision function. A connection to an internal destination is permitted only if the process is bound to a principal and an explicit matching grant exists. In the absence of any grant, the decision function returns *false*, not because a deny rule fires, but because no allow rule matches. External destinations require no grant and are always permitted.

3.5 Decision function

Let $dst = \langle d, t, q \rangle$ denote a connection attempt to destination IP d, transport protocol t, and port q, issued by a process with identifier *pid*. ProcRoute evaluates the following decision function at each `connect()` or UDP `sendmsg()` invocation:

$$allow(pid,\ dst) = \begin{cases} true & \text{if } d \notin \bigcup_{i \in \mathcal{I}} i \text{ (external)} \\ true & \text{if } d \in \bigcup_{i \in \mathcal{I}} i \ \wedge\ prin(pid) \neq \bot \\ & \quad \wedge\ \exists r \in R_{prin(pid)} :\ dst \sqsubseteq r \\ false & \text{otherwise} \end{cases}$$

The function is *total*: every $(pid,\ dst)$ pair yields exactly one verdict. Totality follows from the exhaustive case split on membership in $\mathcal{I}$, the "otherwise" clause, and the totality of *prin* (Section 3), and is a prerequisite for complete mediation (Section 3.9).

When *allow* returns *false*, the kernel returns `EPERM` to the calling process and emits a deny event to the audit log. When it returns *true*, the socket operation proceeds unmodified.

The function is evaluated in the kernel by the eBPF programs attached to the cgroup socket-address hooks, using a sequence of BPF map lookups (Section 3.8).

3.6 Split architecture: client tagging and gateway enforcement

The local decision function *allow* (Section 3.5) mediates every socket operation on a single host. When ProcRoute is deployed across a

WireGuard tunnel, the same cgroup hooks that implement *allow* are reused on the client to resolve $prin(pid)$, but the verdict changes from deny to *tag*: the client stamps the socket with the originating principal, and a cooperating *gateway* enforces a per-peer policy based on the tag and the packet's tunnel source address. This subsection extends the formal model to cover both sides.

Per-peer policy. A gateway serves K WireGuard peers, each identified by its tunnel IP address s_k. The gateway maintains a policy $\Pi_{gw} = \langle \mathcal{I},\ \{s_1, \ldots, s_K\},\ \mathcal{R}_{gw} \rangle$, where $\mathcal{R}_{gw}(s_k, a)$ is the grant set for app_index a on peer s_k. Different peers may have different entitlements: a managed corporate laptop may receive grants for sensitive prefixes that a contractor device does not. The controller distributes $\mathcal{R}_{gw}$ to the gateway, which populates its `app_allow` LPM trie with composite keys that include the peer's tunnel IP (Section 4.4).

Client-side tagging. The client hooks (`cgroup/connect4/6`) resolve $prin(pid)$ as before but *never deny*: instead of returning DENY for unbound processes, the hook stamps the socket with `SO_MARK` encoding the principal identity. Let $tag : PID \to \mathbb{N} \cup \{0\}$ be the tagging function:

$$tag(pid) = \begin{cases} app_index(p) & \text{if } prin(pid) = p \neq \bot \\ 0 & \text{if } prin(pid) = \bot \end{cases}$$

A TC egress BPF program on `wg0` encodes the tag into packet headers: for IPv4, the IP identification field (`ip->id`) carries *app_index* and the TOS byte carries the authorization epoch ε; for IPv6, the flow label carries *app_index* and the traffic class carries ε. The client hooks are purely annotative; they impose no connectivity restrictions.

Gateway-side enforcement. The gateway executes a TC ingress BPF program on its `wg0` interface that evaluates a total function *allow_gw* on every inbound packet before routing. Let s denote the inner source IP (identifying the WireGuard peer) and $a = tag(pkt)$ the app_index extracted from the packet header:

$$allow_gw(pkt,\ dst) = \begin{cases} false & \text{if } a = 0 \text{ (untagged)} \\ true & \text{if } d \notin \bigcup_{i \in \mathcal{I}} i \text{ (external)} \\ true & \text{if } d \in \bigcup_{i \in \mathcal{I}} i \\ & \quad \wedge\ \exists r \in \mathcal{R}_{gw}(s,\ a) :\ dst \sqsubseteq r \\ false & \text{otherwise} \end{cases}$$

The function is total: every packet receives a determinate verdict. Denied packets are dropped via `TC_ACT_SHOT` before reaching the routing layer. Unlike local enforcement, which returns `EPERM` at `connect()` time, gateway-only denies are silent drops; clients observe standard transport timeouts (TCP SYN retransmission, UDP undeliverability) unless local enforcement is also enabled (Section 4.4). Because the grant lookup is keyed on $(s,\ a)$, two peers may carry the same app_index a yet receive different grants.

Tag integrity and trust model. The split architecture relies on three properties:

(1) *Tag authenticity.* WireGuard provides authenticated encryption of the entire inner IP packet. An off-path adversary cannot inject or modify tags in transit.
(2) *Tag binding.* Setting `SO_MARK` on a socket requires `CAP_NET_ADMIN`, which the threat model excludes for the adversary (Section 2). The BPF hooks set the mark in kernel context on behalf of the process; an unprivileged process cannot override it.
(3) *Gateway mediation.* The TC ingress program on `wg0` executes on every packet delivered by the WireGuard interface, before the kernel routes the packet. No bypass path exists for traffic arriving on `wg0`.

End-to-end security property. The composition of client tagging and gateway enforcement preserves the reachable-set guarantee of the local-only model: an unauthorized process ($prin(pid) = \bot$) cannot reach any internal destination via the tunnel, because (i) $tag(pid) = 0$ (untagged), and (ii) the gateway drops all untagged packets. For a bound process with principal p on peer s_k, the gateway allows only destinations covered by $\mathcal{R}_{gw}(s_k,\ app_index(p))$. When the gateway grant set is a subset of the client's local grant set for every principal (the expected configuration, since the gateway may impose additional restrictions), the reachable set under the split architecture is bounded by the local model:

$$Reach_{split}(s_k, p)\ \subseteq\ Reach_{local}(p)\ =\ \bigcup_{r \in R_p} \{dst \mid dst \sqsubseteq r\}$$

3.7 Policy composition and monotonicity

ProcRoute attaches its BPF enforcement program using the `BPF_F_ALLOW_MULTI` flag, which composes programs across the cgroup hierarchy *conjunctively*: the kernel evaluates every attached program from ancestor to descendant, and the connection is allowed only if *all* programs return allow.

Let $allow_A$ denote the decision function of a BPF program attached at ancestor cgroup A, and let $allow_C$ denote the decision function of a supplementary program attached at a descendant cgroup $C \subseteq A$. The effective verdict for a process in C is:

$$allow_{eff}(pid,\ dst) = allow_A(pid,\ dst)\ \wedge\ allow_C(pid,\ dst)$$

Assumption (filter-only programs). cgroup socket-address programs can rewrite the destination address/port in the `bpf_sock_addr` context. ProcRoute's enforcement program is a pure filter and does not modify the destination tuple. The monotonicity claim below is stated for the common deployment in which co-attached descendant programs are also filter-only; under this assumption, reachability is determined solely by the conjunction of allow/deny verdicts.

THEOREM 3.1 (MONOTONICITY OF `BPF_F_ALLOW_MULTI`). *Under conjunctive composition, the set of destinations reachable by a process in descendant cgroup $C \subseteq A$ is a subset of those reachable under the ancestor's policy alone:*

$$\{dst \mid allow_{eff}(pid,\ dst) = true\} \subseteq \{dst \mid allow_A(pid,\ dst) = true\}$$

A program attached at a child cgroup can further restrict but never expand the ancestor's allow set.

PROOF SKETCH. By definition, $allow_{eff} = allow_A \wedge allow_C$. For any (pid, dst) pair where $allow_{eff} = true$, both $allow_A = true$ and $allow_C = true$ must hold (conjunction). Therefore every element of

the effective allow set is in A's allow set, establishing the subset relation. The argument extends by induction to any finite chain of cgroups $C_1 \subseteq C_2 \subseteq \cdots \subseteq A$: each additional conjunct can only shrink the allow set. □

This property is critical for ProcRoute's security posture: the default-deny baseline established by the ancestor program cannot be weakened by any attachment in a descendant cgroup, even if the descendant's program is controlled by a different administrative entity (e.g., an application-specific audit hook added by a team-level policy). The implementation-level implications of this composition model, including the contrast with `BPF_F_ALLOW_OVERRIDE` (which would break monotonicity), are discussed in Section 4.2. Note that if a descendant program rewrites the destination tuple, it can change what the kernel ultimately connects to; such rewriting is orthogonal to (and can undermine) verdict-only monotonicity. ProcRoute is designed and deployed as a non-mutating filter.

Theorem 3.2 (Non-widening policy updates). *Let* $\Pi = \langle \mathcal{I}, \mathcal{P}, \mathcal{R} \rangle$ *denote the current policy and* $\Pi' = \langle \mathcal{I}', \mathcal{P}', \mathcal{R}' \rangle$ *an updated policy installed by the controller via a sequence of* `bpf_map_update_elem` *and* `bpf_map_delete_elem` *calls. If the controller applies updates using a* fail-closed *ordering: (i) for maps whose absence implies* deny *on the internal path (e.g.,* `cgroup_to_app`, `app_allow`, `app_exec_hash`, `task_verified`*), it* deletes *removed entries before* inserting *new entries; and (ii) for the* `internal_prefixes` *classifier (where a miss implies external passthrough* allow*), it* inserts *new prefixes before* deleting *removed prefixes, then at every intermediate map state* M_k *observed by a concurrent BPF hook invocation, the effective allow set is a subset of* $Allow(\Pi) \cup Allow(\Pi')$*:*

$$Allow(M_k) \subseteq Allow(\Pi) \cup Allow(\Pi')$$

In particular, no destination that is denied by both *the old and new policies is transiently permitted during the update.*

Proof sketch. Each map operation is atomic. For *deny-biased* maps (`cgroup_to_app`, `app_allow`, etc.), a missing entry returns deny; deleting before inserting can only shrink the allow set before expanding it, so no transient allow absent from both Π and Π' can appear. For the *allow-biased* `internal_prefixes` map (miss ⇒ external passthrough), inserting before deleting keeps the classified-internal set as a superset of both $\mathcal{I}$ and $\mathcal{I}'$, preventing transient external reclassification. Combining both orderings with the total decision function yields $Allow(M_k) \subseteq Allow(\Pi) \cup Allow(\Pi')$. □

3.8 Enforcement pipeline

The decision function is implemented as a six-stage pipeline executed inside the BPF hook on each socket address operation:

(1) **Destination extraction.** Read the destination IP, port, and protocol from the `bpf_sock_addr` context.
(2) **Internal prefix lookup.** Query the `internal_prefixes` LPM trie with the destination IP. If no match, return allow (external destination; split-tunnel passthrough).
(3) **Principal resolution.** Read the cgroup ID of the calling process and look up `cgroup_to_app`. If no binding exists ($prin = \perp$), return deny.
(4) **Binary hash verification gate** (optional). If the principal's policy includes an `exec_hash` requirement, look up the calling process's TGID in the `task_verified` map. If no entry exists, or the verified principal does not match, return deny. This step is skipped for principals without a declared hash, imposing zero overhead on the common case.
(5) **Per-app prefix lookup.** Construct a composite key $\langle p, d \rangle$ and query the `app_allow` LPM trie. If no matching rule exists, return deny.
(6) **Port and protocol check.** Verify that the destination port and protocol satisfy the matched rule's constraints. If not, return deny; otherwise return allow.

Every deny emits a structured event (PID, command name, cgroup ID, principal index, destination IP, port, protocol, timestamp) to a ring buffer for user-space consumption.

3.9 Security invariants

ProcRoute maintains six invariants, grouped into local enforcement and gateway enforcement:

Local enforcement. (1) **Default-deny**: an internal connection requires both a bound principal and an explicit matching grant; (2) **Unimpeded external access**: destinations outside $\mathcal{I}$ are never blocked; (3) **Complete mediation**: every `connect()`/`sendmsg()` in the ProcRoute subtree is evaluated before the kernel processes the address [14, 15]; (4) **Audit completeness**: every deny emits a structured event.

Gateway enforcement. (5) **Tag integrity**: unprivileged processes cannot forge `SO_MARK` values, because setting the mark requires `CAP_NET_ADMIN` (excluded by the threat model); the BPF hooks set the mark in kernel context and an unprivileged adversary cannot override it. (6) **Gateway mediation**: the TC ingress BPF program on `wg0` evaluates every packet arriving on the tunnel interface before the kernel routes the packet; untagged or policy-violating packets are dropped via `TC_ACT_SHOT`. Failure modes are analyzed in Sections 5.5 and 7.

3.10 Process identity and executable hash verification

Process identity in ProcRoute is determined by cgroup membership: the controller creates a cgroup subtree per principal and moves application processes into it at launch time. The BPF program calls `bpf_get_current_cgroup_id()` to obtain a stable, kernel-assigned identifier and looks it up in `cgroup_to_app`.

Because child processes inherit cgroup membership, a child that `exec()`s a different binary retains the parent's grants by default. To tighten this, the prototype implements optional per-binary SHA-256 hash verification. When a principal's policy includes an `exec_hash` field, a BPF program on the `sched_process_exec` tracepoint invalidates the prior verification and emits an exec event. The user-space daemon reads the event, computes the binary's SHA-256 digest via `/proc/<tgid>/exe`, and on match sets `task_verified[tgid] = app_index`; on mismatch, the process remains denied. A companion tracepoint on `sched_process_exit` deletes the TGID entry,

preventing stale verifications. Connections attempted before verification completes are denied. Principals without a declared hash skip the gate entirely, imposing zero overhead on the common case.

4 Design and Implementation

ProcRoute comprises three components: (1) a *policy authority* on the enterprise control plane that defines internal route sets and per-application grants; (2) an *endpoint controller* (user-space daemon) that manages cgroups, loads BPF programs, populates policy maps, and streams audit logs; and (3) *kernel enforcement* via eBPF programs attached to cgroup socket-address hooks. Figure 1 shows the overall deployment architecture.

4.1 Enforcement hooks and policy maps

ProcRoute attaches eBPF programs to four cgroup v2 socket-address hooks: `connect4/6` and `sendmsg4/6`. These hooks execute synchronously inside the `connect()` (or UDP `sendmsg()`) system-call path, before the kernel constructs or transmits any packet. On deny, the kernel returns `EPERM`; no SYN segment or UDP datagram is emitted.

The BPF program interacts with six map structures to implement the enforcement pipeline:

- `internal_prefixes`: an LPM trie encoding the internal prefix set $\mathcal{I}$. A miss means the destination is external and the connection is allowed without further checks.
- `cgroup_to_app`: a hash map that resolves the calling process's cgroup ID to an application principal. A miss means no principal binding exists; the connection is denied.
- `app_exec_hash`: a hash map keyed by principal index whose values are 32-byte SHA-256 digests. Presence of an entry means the principal requires binary hash verification; no key is present if verification is not required.
- `task_verified`: a hash map keyed by TGID whose value is the verified principal index. Set by the user-space daemon after confirming the binary's SHA-256 hash matches the policy. The BPF hook consults this map only for principals with an entry in `app_exec_hash`.
- `app_allow`: a per-principal LPM trie keyed by (principal, destination) encoding the normalized grant set. Under the well-formedness condition of Section 3.2, at most one rule matches a given destination IP, so a single LPM lookup plus a port/protocol check implements the grant predicate; otherwise deny.

Every deny emits a structured audit event to a ring buffer. A second ring buffer (`exec_events`) carries `sched_process_exec` notifications to user space for hash verification (Section 3.10).

4.2 Attach strategy and composition

ProcRoute attaches its enforcement program to the session-level cgroup using `BPF_F_ALLOW_MULTI`. Under this flag, programs *accumulate* across the cgroup hierarchy: the kernel runs every attached program from ancestor to descendant, and the connection is allowed only if all return allow. A descendant cgroup therefore cannot weaken an ancestor's deny; it can only impose additional restrictions. This provides two properties: (1) *fail-closed*: a process with no principal binding is denied at the parent level before any child program executes; and (2) *composability*: supplementary programs (e.g., per-app audit hooks) can be layered in child cgroups without weakening the baseline deny.

Policy differentiation across applications is achieved not by attaching different BPF programs per child cgroup, but by populating `cgroup_to_app`: each application cgroup's ID maps to a principal whose grant set determines allowed destinations. Processes in cgroups with no map entry receive no grants.

4.3 Controller responsibilities

The user-space controller daemon performs five functions: (1) provisions a cgroup subtree for each application principal defined in the policy; (2) places application processes into their designated cgroups at launch time (or moves them on policy update); (3) loads BPF programs, attaches them, and populates the policy maps including per-principal expected hashes; (4) runs a hash verifier that reads exec events from the `exec_events` ring buffer, computes the SHA-256 digest of the binary, and on match sets the `task_verified` map entry to ungate the process's connections; and (5) drains the deny-events ring buffer and streams structured audit records to a local sink. Map updates take effect atomically from the BPF program's perspective, since each map operation is a single `bpf_map_update_elem` call. In addition, the controller (i) normalizes each principal's grant set to satisfy the destination-disjointness condition of Section 3.2 before populating `app_allow`, and (ii) applies the fail-closed update ordering of Theorem 3.2 (insert-before-delete for `internal_prefixes`, delete-before-insert for deny-biased maps).

4.4 Split architecture: client tagging and gateway enforcement

When ProcRoute protects traffic traversing a WireGuard tunnel, the enforcement boundary splits between the client endpoint and the gateway. The client reuses the same cgroup `connect4/6` hooks described in Section 4.1, but instead of returning DENY for unbound processes, the hooks annotate outbound packets with the originating principal; the gateway then enforces policy on the annotated packets before routing them into the internal network. This subsection describes the four components of the split architecture.

4.4.1 Client-side tagging. The cgroup `connect4/connect6` hooks on the client resolve $prin(pid)$ using the same `cgroup_to_app` map as the local-only mode but instead of denying unbound processes, the hooks stamp the socket with `SO_MARK` encoding the principal's `app_index`. A TC egress BPF program on the `wg0` interface copies the mark into packet header fields: for IPv4, `ip->id` carries `app_index` and `ip->tos` carries the authorization epoch ε; for IPv6, the flow label carries `app_index` and the traffic class carries ε. Unbound processes ($prin = \perp$) receive `SO_MARK = 0`; their packets carry tag zero and are dropped at the gateway. The client hooks never deny: connectivity restrictions are deferred entirely to the gateway, avoiding duplicate enforcement and simplifying client-side auditing.

4.4.2 Gateway-side enforcement. The gateway runs a TC ingress BPF program on its `wg0` interface. The program executes the following steps on every inbound packet, before the kernel routes the packet:

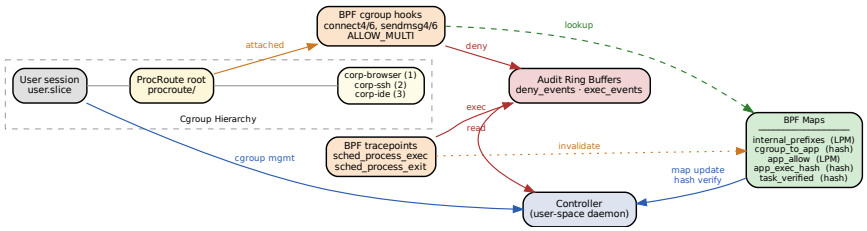

Figure 1: ProcRoute enforcement architecture: controller, BPF hooks, maps, and audit path.

(1) **Tag extraction.** Read `app_index` and epoch from the IPv4 identification / TOS fields (or IPv6 flow label / traffic class). Record the inner source IP s (the peer's tunnel address).
(2) **Untagged drop.** If `app_index` = 0, return `TC_ACT_SHOT`.
(3) **Per-peer, per-principal LPM lookup.** Construct the composite key $\langle s,$ `app_index`, $dst_ip\rangle$ and query `app_allow`. If no match, drop. The three-part key ensures that two peers carrying the same `app_index` are evaluated against their respective grant sets.
(4) **Port and protocol check.** Verify the destination port and protocol against the matched rule's constraints. On mismatch, drop.
(5) **Tag clearing.** On allow, clear the tag fields (reset `ip->id`, `tos`) so that internal hosts see clean headers. Emit a deny event to the ring buffer on drop.

Exempt ports (e.g., WireGuard keepalive, DNS to the gateway's own resolver) are whitelisted before step 2 to avoid breaking control-plane traffic.

4.4.3 Flow cache. To amortize the cost of per-packet LPM lookups on long-lived flows, the gateway maintains an LRU hash map keyed on the 5-tuple (source IP, destination IP, source port, destination port, protocol). The first packet of each flow executes the full lookup pipeline and, on allow, inserts an entry with a 5-second TTL. Subsequent packets matching the cached 5-tuple skip directly to the allow verdict. The cache is invalidated atomically on epoch bump: the program compares the cached epoch against the current global epoch ε and treats stale entries as misses, forcing a fresh lookup. Section 6.3 reports cache hit rates of 75% for short flows and >99.99% for long flows.

4.4.4 Gateway revocation. The authorization epoch ε embedded in each packet header provides a revocation signal. When the controller bumps the epoch (via `bpf_map_update_elem` on the gateway's epoch array), the TC ingress program detects the mismatch between the packet's epoch and the current global epoch and drops the stale packet. Flow-cache entries carry the epoch at insertion time; a stale cache entry is treated as a miss, forcing a fresh LPM lookup against the updated policy maps. This provides sub-millisecond revocation: the median epoch-bump latency measured in Section 6.3 is 190 µs, with new connections blocked within 146 µs.

Tag-carrier limitations. The IPv4 identification field is consumed by IP fragmentation; the TC egress program sets DF and the prototype assumes a tunnel MTU that avoids fragmentation (WireGuard enforces this by default). Overwriting TOS/traffic class clobbers DSCP and ECN bits; the gateway clears tag fields on allow, restoring zeros, which is acceptable for tunnel-interior hops but discards any upstream DSCP markings. The IPv6 flow label is 20 bits, limiting `app_index` to 2^{20} principals, sufficient for endpoint policy but may conflict with ECMP hashing on intermediate routers; the WireGuard outer header carries its own flow label, so inner reuse does not affect outer-path ECMP. A production deployment in a DSCP-sensitive network could avoid header-field overloading entirely by using a dedicated encapsulation shim (e.g., a Geneve TLV or a fixed-offset preamble inside the WireGuard tunnel) at the cost of additional per-packet overhead; the prototype uses header-field tagging because it requires no tunnel-format changes and is sufficient for the evaluated scenarios.

5 Security Analysis

We now analyze the security properties ProcRoute provides under the threat model: enforcement completeness, the least-privilege guarantee of default-deny, the integrity of cgroup-based principal binding, and the boundaries of these guarantees.

5.1 Enforcement completeness at socket operations

ProcRoute interposes on outbound connections via four eBPF cgroup socket-address hooks (`cgroup/connect4/6`, `cgroup/sendmsg4/6`), invoked *before* the kernel processes the socket address [14]. Returning `BPF_DROP` fails the call with `EPERM` before any packet is emitted. All user-space TCP connections pass through `connect()`, and all UDP transmissions through `sendmsg()`-family calls; cgroup BPF programs are inherited by descendant cgroups [15], so the entire ProcRoute subtree is covered without re-attachment. Raw sockets (`SOCK_RAW`) are not mediated but require `CAP_NET_RAW`, excluded by the threat model.

5.2 Default-deny and least privilege

The decision function yields default-deny on internal prefixes: an unbound process ($prin = \perp$) has zero internal access regardless of UID or routing table, and a bound process can reach only its granted

resources R_p. External destinations are never blocked, preserving split-tunnel Internet access.

5.3 Cgroup binding integrity

ProcRoute's security rests on two properties: (1) an unprivileged adversary cannot join an authorized cgroup, and (2) the controller binds only intended processes.

Atomic placement (primary path). On Linux ≥5.7, the prototype uses `clone3()` with the `CLONE_INTO_CGROUP` flag as the *primary, production path* for process placement. The controller passes the target cgroup's file descriptor to `clone3()`, and the kernel places the child into the designated cgroup atomically at creation time, before any user-space code in the child executes. This eliminates the window between process creation and cgroup migration entirely; no interposition gap exists in which the child could issue a `connect()` under the wrong principal.

Forked children inherit cgroup membership; `exec_hash` verification invalidates grants on `exec()`, but `fork()`-only children retain grants unconditionally.

Binding integrity statement. An unprivileged process cannot migrate itself into a ProcRoute application cgroup for three reasons. First, ProcRoute's controller creates all application cgroups under `/sys/fs/cgroup/procroute/` with root ownership (`uid=0`, `gid=0`); writing a PID to `cgroup.procs` requires write permission on that file, which unprivileged processes do not possess [15]. Second, the controller does *not* delegate any ProcRoute cgroup subtree [16]: no `cgroup.delegate` file is set, and no non-root user receives write access to any node in the ProcRoute hierarchy. Third, even if an adversary creates a cgroup elsewhere in the hierarchy (e.g., under a user-delegated subtree), that cgroup has no entry in the `cgroup_to_app` BPF map; the BPF hook resolves $prin = \perp$ and returns DENY. These three properties together ensure that cgroup-based principal binding cannot be forged by an unprivileged adversary.

5.4 Tag integrity in split architecture

When ProcRoute operates in split mode, the security argument extends from cgroup binding integrity to tag integrity across the tunnel.

`SO_MARK` *protection.* The client BPF hooks set `SO_MARK` within the kernel's BPF execution context; the mark is subsequently copied into packet headers by the TC egress program. Setting `SO_MARK` from user space requires `CAP_NET_ADMIN`, which the threat model excludes. An unprivileged adversary therefore cannot forge a mark: its sockets carry `SO_MARK` = 0, producing untagged packets that the gateway drops unconditionally.

Gateway enforcement completeness. The TC ingress program on `wg0` executes on every packet delivered by the WireGuard interface. Because `wg0` is the only ingress path for tunnel traffic, and the TC hook fires before the kernel routes the packet, there is no bypass path. Combined with WireGuard's authenticated encryption, which prevents off-path injection or modification of inner-packet headers, the gateway sees exactly the tags stamped by the client. WireGuard also binds each peer's public key to its allowed tunnel IP, so the inner source address s used in the per-peer grant lookup is cryptographically authenticated; one peer cannot spoof another's tunnel IP.

Composition argument. The combination of (i) mandatory client tagging (unbound processes produce tag = 0), (ii) unforgeability of `SO_MARK` by unprivileged processes, (iii) WireGuard authenticated encryption binding tag to source peer, and (iv) gateway enforcement of a per-peer policy $\mathcal{R}_{gw}(s_k, a)$ keyed on tunnel source IP and app_index implies that the reachable set of any process under the split architecture is bounded by the gateway's grant set for that peer. When the gateway grant set is a subset of the client's local grants (the expected configuration), this gives $Reach_{split}(s_k, p) \subseteq Reach_{local}(p)$ for all peers s_k and principals p (including $\perp$), as stated in Section 3.6.

5.5 Residual attack surface

In-process compromise. Code execution *within* an authorized process (memory corruption, malicious extensions, library injection) inherits the victim's cgroup membership and grants. This is inherent to any process-granularity boundary and requires complementary controls (sandboxing, exploit mitigation). Complementary mechanisms operating within the process boundary, such as seccomp-BPF profiles restricting `connect()` argument ranges, Landlock network rules (Linux ≥6.7), and CFI-instrumented binaries preventing control-flow hijacking, can significantly raise the bar for injected code reaching a socket call. For browser principals specifically, the existing renderer process sandbox already provides an independent enforcement boundary that does not rely on cgroup membership.

Privileged attacker. A `root`/`CAP_SYS_ADMIN` attacker can detach BPF programs or modify maps; the threat model excludes this case.

DNS-based bypasses. ProcRoute enforces on IP prefixes, not DNS names. Resolved IPs outside $\mathcal{I}$ reach no internal resource; those inside $\mathcal{I}$ are subject to the decision function. DNS integrity is an orthogonal concern.

5.6 Policy shrink and revocation behavior

ProcRoute intervenes at `connect()` and `sendmsg()` time; it does not monitor sockets after establishment. Consequently, when a policy update *shrinks* a principal's grant set (e.g., removing a prefix from R_p), TCP connections that were authorized under the prior policy and are already in the `ESTABLISHED` state persist until they are closed by the application or time out. New connections to the revoked prefix are denied immediately by the updated BPF maps, but the in-flight socket retains its kernel state and could continue to exchange data.

The prototype implements G1 (connect-time) and two mechanisms toward G2 (data-path revocation): (1) an *authorization epoch* stored in a BPF array map: on policy update, the controller increments the epoch; each `connect` hook stamps the socket cookie with the current epoch, and each `sendmsg` hook denies sends on stale sockets (immediate UDP revocation); (2) a *background sweeper* that scans `/proc/net/tcp` at `1 s` intervals and issues `ss -kill` (`SOCK_DESTROY`) for stale TCP sockets. A production deployment could add a `sock_ops` hook for per-segment enforcement; We believe the sweeper program is sufficient for the evaluation.

6 Evaluation

ProcRoute's enforcement spans two layers: cgroup socket-address hooks that resolve principals and evaluate grants at `connect()`/`sendmsg()` time, and, in split mode, TC programs that tag and enforce packets across a WireGuard tunnel. We evaluate these layers separately and in composition. Single-machine loopback microbenchmarks (Q1–Q6) isolate the cgroup hook overhead that is common to both local-only and client-side tagging modes, independent of network and encryption costs. A two-machine WireGuard deployment then measures the end-to-end cost of the full stack (client tagging, tunnel encryption, and gateway TC enforcement) including throughput, latency, multi-stream scaling, flow-cache effectiveness, gateway policy scaling, and revocation.

6.1 Experimental setup

Single-machine microbenchmark (loopback). All Q1–Q6 benchmarks ran on a dedicated bare-metal server: AMD EPYC 7443P (24 cores, 2.85 GHz base), 64 GB DDR4, Ubuntu 24.04.3 LTS, cgroup v2 unified hierarchy. Clients and listeners were pinned to separate cores. Internal-prefix traffic used a loopback alias, causing the BPF hook to walk the full pipeline, thus isolating the cgroup BPF hook overhead from network and encryption costs.

WireGuard deployment (two machines). Each machine is an AMD Ryzen 7 7700X (8 cores/16 threads, 4.5 GHz, 61 GB RAM) running Ubuntu 24.04 LTS, connected via a WireGuard tunnel over public IPv4. On the *client*, cgroup `connect4/6` hooks tag authorized flows with `SO_MARK` and IPv6 flow labels. On the *gateway*, a TC ingress BPF program on `wg0` enforces per-principal policy with an optional flow cache. Five configurations are tested: *wg_baseline* (no BPF), *wg_nftables* (nftables cgroup match), *wg_tag_only* (client hooks only), *wg_enforce_nocache* (full enforcement, no cache), and *wg_enforce_cache* (full enforcement with flow cache).

6.2 Single-machine microbenchmark summary

To isolate BPF hook overhead from network and encryption costs, we ran six microbenchmark experiments (Q1–Q6) on a loopback path using a dedicated bare-metal server (AMD EPYC 7443P, 24 cores, 64 GB RAM). Table 1 summarizes the key results; full methodology, per-experiment tables, and analysis are in Appendix A.

The loopback results confirm that BPF hook overhead is negligible (2.7 µs on the internal-allow path), flat across policy sizes from 4 to 4 096 prefixes and 5 to 200 principals, and structurally safe during policy reloads. ProcRoute blocked all 82 pivot attempts while retaining full external connectivity, recorded zero transient allows across 1.24 million connection attempts during policy reload, and revocation is immediate for UDP and sub-15 ms for TCP.

These cgroup hook costs carry directly into the split architecture: the client-side `connect4/6` hooks execute the same principal-resolution and `SO_MARK` stamping logic, so the 2.7 µs overhead measured here is the per-connection cost on the client in split mode as well. The WireGuard evaluation below adds the TC tagging and gateway enforcement layers on top of this baseline.

Table 1: Single-machine microbenchmark summary (loopback). Full details in Appendix A.

Experiment	Key result
Q1: Hook overhead	p50 internal-allow = 25.6 µs (+2.7 µs); throughput = 31.2 Gbps
Q2: Pivot prevention	82/82 unauthorized attempts blocked
Q3: Policy scaling	Flat p50 at 28.7 µs with 4 096 prefixes
Q4: Update safety	0 transient allows / 1.24M attempts
Q5: nftables comparison	ProcRoute 25.6 µs vs. nftables 30.3 µs
Q6: Revocation	UDP epoch deny ~27 µs; TCP sweeper ~11 ms

Table 2: WireGuard single-stream throughput and connect latency (10 trials each).

Config.	Tput (Mbps)	p50 (µs)	p95 (µs)	p99 (µs)
wg_baseline	8 878	89.8	98.7	100.9
wg_nftables	7 836	97.3	100.6	101.9
wg_tag_only	8 878	90.3	98.4	100.5
wg_enforce_nocache	8 878	93.4	99.0	100.7
wg_enforce_cache	8 876	93.7	101.2	102.3

6.3 End-to-end WireGuard deployment

This section evaluates ProcRoute over a realistic two-machine WireGuard tunnel similar to an enterprise deployment.

6.3.1 Throughput and connect latency. Table 2 summarizes single-stream TCP throughput (10 s `iperf3`, 10 trials) and TCP `connect()` latency over the WireGuard tunnel. All ProcRoute configurations match the WireGuard baseline at 8 878 Mbps. The nftables cgroup-matching configuration achieves only 7 836 Mbps, 12% lower than baseline, due to per-packet netfilter rule evaluation traversal on the gateway. Connect latency at p50 is 89.8 µs for the baseline and 93.4–93.7 µs for full enforcement (+3.6 µs), well within the WireGuard round-trip noise floor. The nftables configuration adds 7.5 µs.

6.3.2 Multi-stream scaling. Table 3 reports aggregate throughput with 8 and 16 parallel `iperf3` streams. At 16 streams ProcRoute enforcement without cache achieves 8 298 Mbps, within 2.3% of the baseline; with the flow cache the gap is 2.8%. The nftables configuration drops to 7 661 Mbps, 9.8% below baseline, confirming that per-packet netfilter rule evaluation overhead compounds unfavorably with stream count, which is critical in enterprise environments.

6.3.3 Flow-cache effectiveness. The gateway TC program optionally caches per-flow allow decisions keyed on (src_ip, dst_ip, dst_port, protocol) so that subsequent packets skip the full LPM + principal lookup. Hit rates vary by workload: many short flows (1 000 four-packet flows) yield 75% cache hits, while few long flows (4 persistent streams) reach >99.99% (650 908 hits vs. 4 misses). For long flows the cache amortizes nearly all gateway lookups; for short

Table 3: WireGuard multi-stream throughput (Mbps, 10 trials).

Configuration	8 streams	16 streams
wg_baseline	8 786	8 497
wg_nftables	7 780	7 661
wg_tag_only	8 780	8 439
wg_enforce_nocache	8 757	8 298
wg_enforce_cache	8 764	8 260

Table 4: Gateway policy scaling (3 trials).

Principals	Total prefixes	Startup (ms)	Tput (Mbps)
10	100	204.4	8 858
50	500	204.5	8 856
100	1 000	204.6	8 858
250	2 500	204.3	8 857
500	5 000	204.8	8 855

Table 5: WireGuard revocation latency (10 trials).

Metric	Median (µs)	Range (µs)
Epoch bump	190	189–195
New connection block	146	137–153
Steady-state connect	102	94–109

flows the first packet incurs a full lookup but subsequent packets hit the cache.

6.3.4 Gateway policy scaling. We varied the number of principals (10–500) with 10 prefixes each and measured gateway startup time and sustained throughput (Table 4). Startup time is flat at ∼204 ms across all sizes; throughput remains at 8 855–8 858 Mbps regardless of whether the policy contains 100 or 5 000 total prefixes. The LPM trie lookup is O(prefix length) and independent of map population, consistent with the flat scaling observed in the single-machine results (Table 8).

6.3.5 WireGuard revocation. Table 5 reports revocation latency over the WireGuard tunnel. Epoch bumps complete in 190 µs (median), new connections are blocked within 146 µs, and steady-state connect latency is 102 µs, all sub-millisecond. Flow-cache entries are invalidated atomically on epoch bump, confirming that cached allow decisions do not survive revocation.

6.3.6 Summary. The two-machine WireGuard results confirm that ProcRoute's architecture scales from single-machine loopback to a realistic encrypted tunnel with negligible overhead. Client-side cgroup hooks add 3.6 µs to connect latency; gateway-side TC enforcement has no measurable throughput impact. In contrast, the nftables cgroup-matching configuration incurs a 12% throughput penalty, validating ProcRoute's design choice of flow-level tagging over per-packet rule evaluation. Revocation remains sub-millisecond, and policy scaling is flat to 5 000 prefixes, consistent with the single-machine results.

7 Discussion

7.1 Deployment model

ProcRoute can be deployed incrementally: (1) observe-only mode to log which processes attempt internal access; (2) enforce for high-risk prefixes first; (3) expand per-application allowlists as needed. For HTTP/S applications already mediated by a ZT proxy, ProcRoute restricts direct IP access to internal subnets while allowing proxy endpoints. For non-HTTP protocols, ProcRoute provides least-privilege route gating without protocol translation.

The WireGuard evaluation demonstrates that ProcRoute's split client/gateway architecture operates at line rate over an encrypted tunnel, with negligible connect-latency overhead and flat policy scaling to 5 000 prefixes. This confirms that the deployment model is practical for production quality deployments in enterprise environments. Although the prototype and evaluation use WireGuard as the tunnel substrate, the split architecture is not WireGuard-specific: it requires only (i) a kernel-visible tunnel interface on which a TC program can inspect and tag packets, and (ii) authenticated encryption that binds inner headers to a peer identity. Any VPN or overlay that exposes a `tun`/`tap` interface (e.g., OpenVPN, IPsec `xfrm` interfaces, Tailscale, Cloudflare WARP) satisfies both conditions, and the same TC ingress/egress programs could be attached with only interface-name and peer-identity mapping changes.

7.2 Future work

We see three natural extensions: automatic policy synthesis from observed traffic, DNS-name-aware selectors to reduce over-broad prefix grants, and cross-platform prototypes on Windows (WFP callout drivers) and macOS (Network Extension content filters). ProcRoute's access-control model is OS-agnostic; the decision function requires only a principal-binding mechanism, a synchronous enforcement hook, and an audit channel, all available on Windows and macOS.

8 Related Work

nftables cgroup matching. nftables `socket cgroupv2` matching [10] provides basic per-cgroup destination filtering. ProcRoute contributes a formal model, unified policy surface, monotonic composition (Theorem 3.1), and structured audit.

systemd `IPAddressAllow/Deny`. systemd (v235+) attaches eBPF programs to a unit's cgroup for IP-prefix filtering [13], limited to long-running services with static prefixes, and unable to cover interactive desktop applications, dynamic prefix changes, or cross-unit policy.

Process confinement. SELinux [6], AppArmor [2], Capsicum [1], `seccomp` [17], and Landlock [12] confine processes via labels, descriptors, or syscall classes but do not express destination-prefix policies for dynamic split-tunnel route sets.

Zero trust and eBPF ecosystem. ZT architectures [11, 18] authenticate users/devices; ProcRoute adds process-level enforcement. eBPF projects (Cilium [3], Tetragon [4]) target containers, not endpoint route authorization.

9 Conclusion

Enterprises rely on split tunneling for good reason, but it turns every device into a potential pivot. Existing OS mechanisms like nftables cgroup matching and systemd `IPAddressAllow` can approximate per-process filtering, yet they remain low-level primitives with no formal policy semantics, no composition guarantees, and no structured audit trail. ProcRoute fills this gap: by modeling routes as protected resources and processes as principals governed by a total decision function, organizations can preserve split-tunnel usability while confining internal-route access to authorized processes with auditable, formally grounded guarantees. Our evaluations confirm that ProcRoute matches baseline throughput, adds negligible connect latency, scales flat to thousands of prefixes, and revokes access in sub-millisecond time, all without per-packet rule evaluation overhead.

References

[1] Jonathan Anderson, Robert N. M. Watson, and Kris Kennaway. 2010. Capsicum: Practical Capabilities for UNIX. In *Proc. 19th USENIX Security Symposium*. 29–45.
[2] Canonical Ltd. 2024. AppArmor. https://apparmor.net.
[3] Isovalent. 2024. Cilium: eBPF-based Networking, Observability, and Security. https://cilium.io.
[4] Isovalent. 2024. Tetragon: eBPF-based Security Observability and Runtime Enforcement. https://tetragon.io.
[5] Joint Task Force. 2020. *Security and Privacy Controls for Information Systems and Organizations*. Technical Report SP 800-53 Rev. 5. NIST. doi:10.6028/NIST.SP.800-53r5
[6] Peter Loscocco and Stephen Smalley. 2001. Integrating Flexible Support for Security Policies into the Linux Operating System. In *Proc. USENIX Annual Technical Conference (FREENIX Track)*. 29–42.
[7] MITRE. 2024. Lateral Movement – MITRE ATT&CK. https://attack.mitre.org/tactics/TA0008/.
[8] National Institute of Standards and Technology. 2016. *Guide to Enterprise Telework, Remote Access, and Bring Your Own Device (BYOD) Security*. Technical Report SP 800-46 Rev. 2. NIST. doi:10.6028/NIST.SP.800-46r2
[9] National Security Agency and Cybersecurity and Infrastructure Security Agency. 2021. *Selecting and Hardening Remote Access VPN Solutions*. Technical Report. NSA/CISA. U/OO/184997-21.
[10] netfilter project. 2024. Matching packet metainformation — nftables wiki. https://wiki.nftables.org/wiki-nftables/index.php/Matching_packet_metainformation.
[11] Scott Rose, Oliver Borchert, Stu Mitchell, and Sean Connelly. 2020. *Zero Trust Architecture*. Technical Report SP 800-207. NIST. doi:10.6028/NIST.SP.800-207
[12] Mickaël Salaün. 2017. Landlock: Unprivileged Access Control. In *Proc. Linux Security Summit*.
[13] systemd project. 2024. systemd.resource-control — Resource Control Unit Settings. https://www.freedesktop.org/software/systemd/man/systemd.resource-control.html. Documents `IPAddressAllow=` and `IPAddressDeny=` directives; available since systemd 235.
[14] The Linux Kernel Documentation. 2024. BPF cgroup Programs. https://docs.kernel.org/bpf/.
[15] The Linux Kernel Documentation. 2024. Control Group v2. https://docs.kernel.org/admin-guide/cgroup-v2.html.
[16] The Linux Kernel Documentation. 2024. Control Group v2 — Delegation. https://docs.kernel.org/admin-guide/cgroup-v2.html#delegation.
[17] The Linux Kernel Documentation. 2024. Seccomp BPF (SECure COMPuting with filters). https://docs.kernel.org/userspace-api/seccomp_filter.html.
[18] Rory Ward and Betsy Beyer. 2014. BeyondCorp: A New Approach to Enterprise Security. In *;login: The USENIX Magazine*, Vol. 39. 6–11.
[19] Avishai Wool. 2004. A Quantitative Study of Firewall Configuration Errors. *Computer* 37, 6 (2004), 62–67. doi:10.1109/MC.2004.2
[20] Avishai Wool. 2010. Trends in Firewall Configuration Errors: Measuring the Holes in Swiss Cheese. *IEEE Internet Computing* 14, 4 (2010), 58–65. doi:10.1109/MIC.2010.29

Table 6: TCP `connect()` latency (µs). Δp50 = condition − baseline.

Condition	p50	Δp50	p90	p99
Baseline	22.9	—	24.8	32.6
External-miss	24.7	1.8	27.1	35.1
Internal-allow	25.6	2.7	27.2	34.9

Table 7: Pivot-prevention test matrix (unauthorized process, all denied).

Service	Target subnet	#	Proto
SSH (T1021.004)	10.0.0.0/24, 10.250.0.0/24	30	TCP
HTTPS (T1071.001)	10.0.0.0/24	20	TCP
RDP (T1021.001)	10.0.0.0/24	10	TCP
PostgreSQL (T1210)	10.0.0.0/24, 10.0.1.0/24	10	TCP
Alt-HTTP	10.0.0.0/24	10	TCP
DNS (internal)	10.0.0.0/24, 10.0.1.0/24	2	UDP
Total		**82**	

A Single-Machine Microbenchmark Details

This appendix provides the full methodology and results for the six single-machine microbenchmarks summarized in Section 6.2. All experiments ran on the loopback setup described in Section 6.1.

A.1 Q1: Connection-time enforcement overhead

We measured TCP `connect()` latency under three conditions (Table 6): (1) *baseline*, outside ProcRoute (no hook); (2) *external-miss*, single LPM miss (`127.0.0.1`); (3) *internal-allow*, full pipeline (LPM hit → cgroup-to-principal → per-app LPM → port check). The BPF-only deny overhead (5.7 µs, no TCP handshake) is reported separately in the nftables comparison (Section A.5).

Results. Table 6 reports end-to-end `connect()` latency and isolates BPF overhead (Δp50 = condition p50 − baseline p50). The external-miss path adds 1.8 µs (one LPM lookup that misses). The internal-allow path adds 2.7 µs (three map lookups plus port/protocol check). Tail latencies (p90, p99) are tight on bare metal: the baseline p99 is 32 µs.

TCP throughput was 31.2 Gbps (internal-allow) vs. 34.2 Gbps baseline (9% overhead); the hook fires once at `connect()` time and imposes no per-packet cost.

A.2 Q2: Pivot prevention

All 82 connection attempts from an unauthorized process (no principal binding) across six service categories were denied; Table 7 breaks down the test matrix.

A.3 Q3: Policy-update latency and scaling with policy size

ProcRoute's lifecycle layer (Section 4.1) claims that policy changes take effect by atomically repopulating BPF maps via `bpf_map_update_elem`. We evaluate two aspects: (A) whether map-population

Table 8: Policy-update latency and internal-allow lookup p50 vs. number of internal prefixes (N).

N (prefixes)	Update (ms)	Lookup p50 (µs)
4	107.0	25.5
16	107.1	26.1
64	107.1	26.4
256	107.3	26.2
512	107.5	27.3
1 024	107.4	26.7
2 048	107.2	28.4
4 096	107.2	28.7

Table 9: Transient allows during 10 policy-reload cycles.

Mechanism	Total attempts	Transient allows
ProcRoute (BPF attached)	1 240 367	0
nftables (flush + reload)	3 485	3 085

time remains practical as policy size grows, and (B) whether lookup latency in the data-plane hook degrades with larger maps.

Methodology. Synthetic policies with $N \in \{4 \ldots 4096\}$ prefixes and $M \in \{5 \ldots 200\}$ principals (48 configurations). (A) Update latency: wall-clock time from daemon launch to all maps populated (3 trials). (B) Lookup latency: 2 000 TCP `connect()` iterations (200 warmup) per configuration (3 trials). Table 8 reports medians for $M = 5$.

Results. Update latency is constant at ~107 ms across all configurations from 4 to 4 096 prefixes, dominated by fixed costs (BPF loading, map creation, cgroup provisioning); the per-entry insertion cost is negligible. Internal-allow lookup p50 is flat at 25–29 µs from $N = 4$ to $N = 4096$, confirming that LPM trie lookup time depends on prefix length (at most 32 bits for IPv4), not on the number of entries. Varying the number of principals from 5 to 200 had no measurable effect on lookup latency (all clean medians within 25–29 µs). These latencies are well within the budget of a VPN route renegotiation.

A.4 Q4: Update safety under load

A tight `connect()` loop from an unauthorized process ran through 10 policy-reload cycles, comparing ProcRoute (BPF stays attached; map entries updated in place) with nftables (`nft flush ruleset && nft -f`).

Results. ProcRoute recorded zero transient allows across 1 240 367 attempts (Table 9). nftables' `flush ruleset` reverts to default-accept: 3 085 of 3 485 attempts (88.5%) succeeded during the window, confirming the structural difference stated in Theorem 3.2.

A.5 Q5: Structural comparison with nftables

We implemented an equivalent nftables configuration (nine `socket cgroupv2` rules mirroring the five-principal policy) and ran the same latency microbenchmark; nftables also denied all 82 pivot attempts (Table 7) with comparable median latency (Table 10).

Table 10: p50 TCP `connect()` latency (µs): nftables vs. ProcRoute. † nftables `drop` causes a TCP timeout, not `EPERM`.

Condition	nftables	ProcRoute
Baseline	28.6	22.9
External-miss	29.7	24.7
Internal-allow	30.3	25.6
Internal-deny	†	5.7

Table 11: Revocation latency and epoch-check overhead.

Mechanism	Metric	Value
UDP epoch	First deny	~27 µs
UDP epoch	Denied/total	1 000/1 000
TCP sweeper	50-sock revoke	~11 ms
Epoch ovhd.	p50/p90/p99	24.5/27.5/42.9 µs

Structural differences. (1) nftables evaluates rules in a linear chain where ordering determines the verdict; ProcRoute's conjunctive composition eliminates rule-ordering failures by construction. (2) ProcRoute's deny events include PID, command name, and principal; nftables logs only packet metadata. (3) ProcRoute optionally gates on per-binary SHA-256 verification; nftables has no binary-identity mechanism.

A.6 Q6: Revocation latency

Table 11 summarizes the two revocation mechanisms. After an epoch increment, all 1 000 subsequent UDP `sendto()` calls were denied within ~27 µs. The TCP sweeper terminated 50 established connections in ~11 ms. Epoch-check overhead is negligible (+1.6 µs over baseline).

Limitations. The current epoch mechanism has two limitations: (1) the UDP epoch check applies to all sockets stamped at an older epoch, not only those whose specific grant was revoked; a selective per-principal epoch would reduce false revocations in multi-principal updates; (2) TCP revocation relies on a userspace sweep rather than an in-kernel `sock_ops` hook, bounding the worst-case revocation delay to the sweep interval (1 s in the prototype). A production system could combine both: the epoch mechanism for immediate UDP/first-packet enforcement and a `sock_ops` program for per-segment TCP enforcement. The destination-disjoint assumption (Section 3.2) prevents a single principal from holding overlapping prefixes with different port constraints, e.g., allowing `10.0.0.0/8` on port 443 while restricting `10.1.0.0/16` to port 22. Enterprise firewall audits suggest that 15–25 % of zone-based rulesets contain such overlapping-prefix, port-differentiated entries [19, 20]. This restriction could be lifted by replacing the single LPM lookup with a *cascaded trie*: for each principal, a longest-match hit is followed by a secondary hash-map lookup keyed on $\langle prefix, proto \rangle$, adding one extra map access (~40 ns) per connection but supporting arbitrary per-prefix port predicates.